LoRa®-Based Messaging with MKR WAN 1310

Learn how to use the Serial Monitor to send messages between two MKR WAN 1300 boards using LoRa® technology.

Author · Karl Söderby Last revision · 02/28/2025

In this tutorial, we will use two MKR WAN 1310's to set up a simple message service over a LoRa®-based communication channel. This communication will be achieved through the Serial Monitor, where you can send and receive messages directly.

We will use the **LoRa** library to for the communication, and we will not use any external services. Additionally, we will also create specific addresses for each board. This will help ensure that the messages that we send and receive are only displayed on the corresponding devices.

Special thanks to Sandeep Mistry for creating the LoRa library.

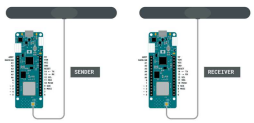
Hardware & Software Needed

- 2x Arduino MKR WAN 1310 (link to store)

- 2x Micro USB cable
- 2x Computers
- Arduino IDE (offline and online versions available)
- Arduino SAMD Board Package installed, **follow this link for instructions**
- **LoRa** library installed, see the **github repository**

Circuit

Follow the wiring diagrams below to create the circuits for the sender and receiver boards.



Sender & Receiver circuit with antenna.

Let's Start

In this tutorial, we will create a message service that uses a LoRa®-based communication channel. In our other tutorials for the MKR WAN 1310 board, we have typically set up one board as a sender, and one as a receiver. Now, we will instead set them up as **both sender and receiver**. This will allow us to both send and receive packets simultaneously, which works very similar to any messenger service you might be used to!

To do this, we basically only need

1310 boards, with only some minor adjustments made in the code for each.

In the code, we will have to do the following to make it work:

- Initialize the **SPI** and **LoRa** libraries.
- Create a string to store outgoing messages.
- Create two bytes: one for local address, one for the destination address.
- Set the radio frequency to 868E6 (Europe) or 915E6 (North America).
- Create a while loop to read input from the Serial Monitor.
- After input is read, create and send a LoRa® packet to destination address.
- Continuously listen for incoming LoRa® packets.
- If a packet comes in that matches the local address, parse it and print it in the Serial Monitor.
- Print the signal strength (RSSI) each time a packet comes in.

Creating the Program

1. First, let's make sure we have the drivers installed. If we are using the Cloud Editor, we do not need to install anything. If we are using an offline editor, we need to install it manually. This can be done by navigating to **Tools > Board > Board Manager...**. Here we need to look for the **Arduino**

2. Now we need to download the **LoRa** library from this repository, where you can install it by navigating to **Sketch > Include Library > Add .ZIP Library...** in the offline IDE.

Code Explanation

We are going to program two separate MKR WAN 1310's in this tutorial. The sketches are 99% identical, but there are two major things that will differ: the `localAddress` and `destination` bytes will be switched on the opposite boards. For personalization, we also added two names to identify who is who in the chat: **Peter** and **Juan**.

The table below provides a better explanation to this:

Name	Local Address	Destination Address
Peter	0xBB	0xFF
Juan	0xFF	0xBB

> *Note: This section is optional and only explains the code. To find the full version of the code, you can find it below this section.*

Programming First Device (Peter)

In the initialization we will include the **SPI** and **LoRa** libraries. We will also create a string named

We will also create two bytes: `localAddress` and `destination`. As mentioned above, these hold the addresses `0xFF` and `0xBB`.

> *Important: these addresses need to be switched on the sketches we upload to the board.*

```
#include <SPI.h>
#include <LoRa.h>

String message;

byte localAddress = 0xB
byte destination = 0xFF
```

In the `setup()` we will begin serial communication and initialize the **LoRa** library.

```
void setup() {
  Serial.begin(9600);
  Serial.println("LoRa
  if (!LoRa.begin(868E6
    Serial.println("Sta
    while (1);
  }
  delay(1000);
}
```

In the `loop()` we will begin by creating a while loop. Whenever we write a message in the Serial Monitor, we simply read it, and store it in the `message` string.

Then, as the message is entered,

longer than 0 characters, we print `"Peter: "` + `message` in the Serial Monitor. This gives us feedback on the message we just wrote, which is a good way to know it has been registered. After that, begin creating the LoRa® packet. Here, we first print the `destination` and `localAddress`, to indicate where the packet is going, and where it is coming from. Then, we print the same information, and send off the packet by using `LoRa.endpacket()`. Here we also reset the `message` string.

Now we also want to receive packets. This is done by calling the function `onReceive(LoRa.parsePacket());`, which will be explained in the next section.

```
void loop() {
  while (Serial.availa
    delay(2);  //delay
    char c = Serial.re
    message += c;
  }

  if (message.length()
    Serial.println("Pe
    LoRa.beginPacket()
    LoRa.write(destina
    LoRa.write(localAd
    LoRa.print("Peter:
    LoRa.endPacket();
    message = "";
  }

  onReceive(LoRa.parse

}
```

Whenever the `onReceive()` function is called upon, it first checks whether a packet has come in or not. If no packet has come, it simply returns to the loop.

But if a packet comes in, there are two major things that happen. First, we read the packet, using the command `int recipient = LoRa.read();`, which contains the `localAddress` (sent from the other board). We then create a string called `incoming`, which we then store the incoming message in.

We then compare `recipient` to `localAddress` and `0xFF`, and if it doesn't match, we print "This message is not for me" in the Serial Monitor. As there are many people using the LoRa®-based network, we might intercept other messages, and if we do, that is the message we will see instead.

Finally, we print the message stored in the `incoming` string in the Serial Monitor, along with RSSI.

```
void onReceive(int pac
  if (packetSize == 0)

  int recipient = LoRa
  String incoming = ""

  while (LoRa.availabl
    incoming += (char)
  }

  if (recipient != loc
    Serial.println("Th
    return;
  }

  Serial.print(incomin
  Serial.print(" || RS
  Serial.println(LoRa.
  Serial.println();
}
```

Programming Second Device (Juan)

As mentioned earlier, the code is almost identical for both devices. The only difference is that we will switch the address in the initialization, and the name printed in the Serial Monitor.

```
#include <SPI.h>
#include <LoRa.h>

String message;

byte localAddress = 0
byte destination = 0x

void setup() {
  Serial.begin(9600);
  Serial.println("LoR
  if (!LoRa.begin(868
    Serial.println("S
    while (1);
  }
  delay(1000);
}
void loop() {
  while (Serial.avail
    delay(2);  //dela
    char c = Serial.r
    message += c;
  }

  if (message.length(
    Serial.println("J
    LoRa.beginPacket(
    LoRa.write(destin
```

Complete Code

If you choose to skip the code building section, the complete code can be found below:

Device #1 Code

```
#include <SPI.h>
#include <LoRa.h>

int counter = 0;
int button = 2;
int buttonState;

void setup() {
  pinMode(button, INP

  Serial.begin(9600);

  while (!Serial);
  Serial.println("LoR

  if (!LoRa.begin(868
    Serial.println("S
    while (1);
  }
  delay(1000);
}

void loop() {
  buttonState = digit

  if (buttonState ==
    // send packet
    LoRa.beginPacket(
```

Device #2 Code

```
#include <SPI.h>
#include <LoRa.h>

String contents = "";
String buttonPress =
bool x;

int led = 2;

void setup() {

  pinMode(led, OUTPUT
  Serial.begin(9600);
  while (!Serial);
  //Wire.begin();
  Serial.println("LoR

  if (!LoRa.begin(868
    Serial.println("S
    while (1);
  }
}

void loop() {
  // try to parse pac
  int packetSize = Lo
  if (packetSize) {
    // received a pac
```

Upload Sketch and Testing the Program

Once we are finished with the code, we can upload the sketches to each board. At this point, we will need **two computers**, as we are going to exchange messages between them. When the code has been uploaded, **open the Serial Monitor on each computer**.

If everything goes right, we should be able to write messages over

in the Serial Monitor of either device, and hit "enter" once finished. This will store the entered message in a string called `message`. In the code, we also created a packet and printed `message` to it. This is done automatically after we have hit "enter", and should now be sent to the other device.

> *Important: the Serial Monitor needs to be open for both devices in order to send and receive messages. If we send a message from Device #1, we will need to have the Serial Monitor open on Device #2.*

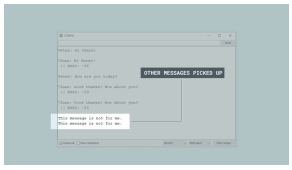
We should now receive the message in Device #2, along with the RSSI and the name of the sender. As chosen in this tutorial, the name for Device #1 is **Peter** and for Device #2, the name is **Juan**.

Sending messages between boards, Serial Monitor.

It also happens that we pick up messages that were not intended for us. Earlier in the sketch, inside the `receive()` function, we used the following command to handle these messages.

```
if (recipient != localA
    Serial.println("Thi
    return;
  }
```

And this is how it looks like in the Serial Monitor:



Other messages picked up.

Troubleshoot

If the code is not working, there are some common issues we might need to troubleshoot:

- Antenna is not connected properly.
- The radio frequency is wrong. Remember, 868E6 for Europe and 915E6 for Australia & North America.
- We have not opened the Serial Monitor.
- We are using the same computer for both boards without a serial interfacing program.

Conclusion

In this tutorial, we have created a LoRa®-based messaging service application, using two MKR WAN 1310 boards and two antennas. In the right conditions, these boards can send messages over very long

solution for remote places where internet access is limited.

Trademark Acknowledgments

LoRa® is a registered trademark of Semtech Corporation.

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.

Need support?

Help Center
Ask the Arduino Forum
Discover Arduino Discord

License

Was this article helpful?